

Het integreren van Python modules in een .NET rekenmodel

Een proof-of-concept bij ILVO

Caradoc Roetynck

caradoc.roetynck@student.hogent.be

Promotor: Jan Willem

Co-promotor: Samuel Bosch (ILVO)

Hogeschool Gent, Valentin Vaerwyckweg 1, 9000 Gent

Samenvatting

Bij ILVO (Instituut voor Landbouw-, Visserij- en Voedingsonderzoek) ontwerpen onderzoekers complexe rekenmodellen die door IT-developers in .NET worden geïmplementeerd. Hierdoor kunnen onderzoekers hun modellen niet altijd zelf aanpassen of snel testen, wat de experimenteerruimte beperkt. Dit onderzoek toont aan dat door de integratie van Python met de bestaande C#-code, onderzoekers de autonomie krijgen om modellen direct te valideren en aan te passen, zonder de stabiliteit van de hoofdapplicatie te schaden. Naast de Python gaat dit onderzoek ook diep in Clean Code en Documentatie Drift, aangezien dit belangrijke zaken zijn om code te verduidelijken.

Keuzerichting: Software Development

Sleutelwoorden: Docker, C#, Jupyter-Notebooks, Razor

Broncode:

1. Introductie

Bij ILVO vormen complexe rekenmodellen de kern van onderzoek naar milieu-impact en emissies in de landbouw. Deze modellen worden door onderzoekers bedacht, maar door IT-developers vertaald naar software in een .NET/C# omgeving.

De Uitdaging

Hoewel deze taakverdeling logisch is, zorgt het in de praktijk voor een kloof tussen de wetenschappelijke theorie en de technische praktijk:

- Beperkte Zichtbaarheid: De wetenschappelijke logica zit verweven in de broncode, waardoor de berekeningsstappen voor onderzoekers moeilijk te verifiëren zijn.
- Afhankelijkheid: Voor elke wijziging of test van een formule is technische hulp nodig, wat de snelheid van het onderzoek vertraagt.

Onderzoeksvraag

Hoe kunnen we de kloof tussen IT-logica en wetenschappelijke analyse verkleinen, zodat onderzoekers zonder diepgaande programmeerkennis inzicht krijgen in hun modellen en er zelf mee kunnen experimenteren?

2. Methodologie & Technologie

Om deze kloof te dichten, is een hybride architectuur ontwikkeld die onderzoekers in staat stelt om direct met de gecompileerde .NET-logica te interageren via een vertrouwde Python-omgeving.

Gekozen Aanpak:

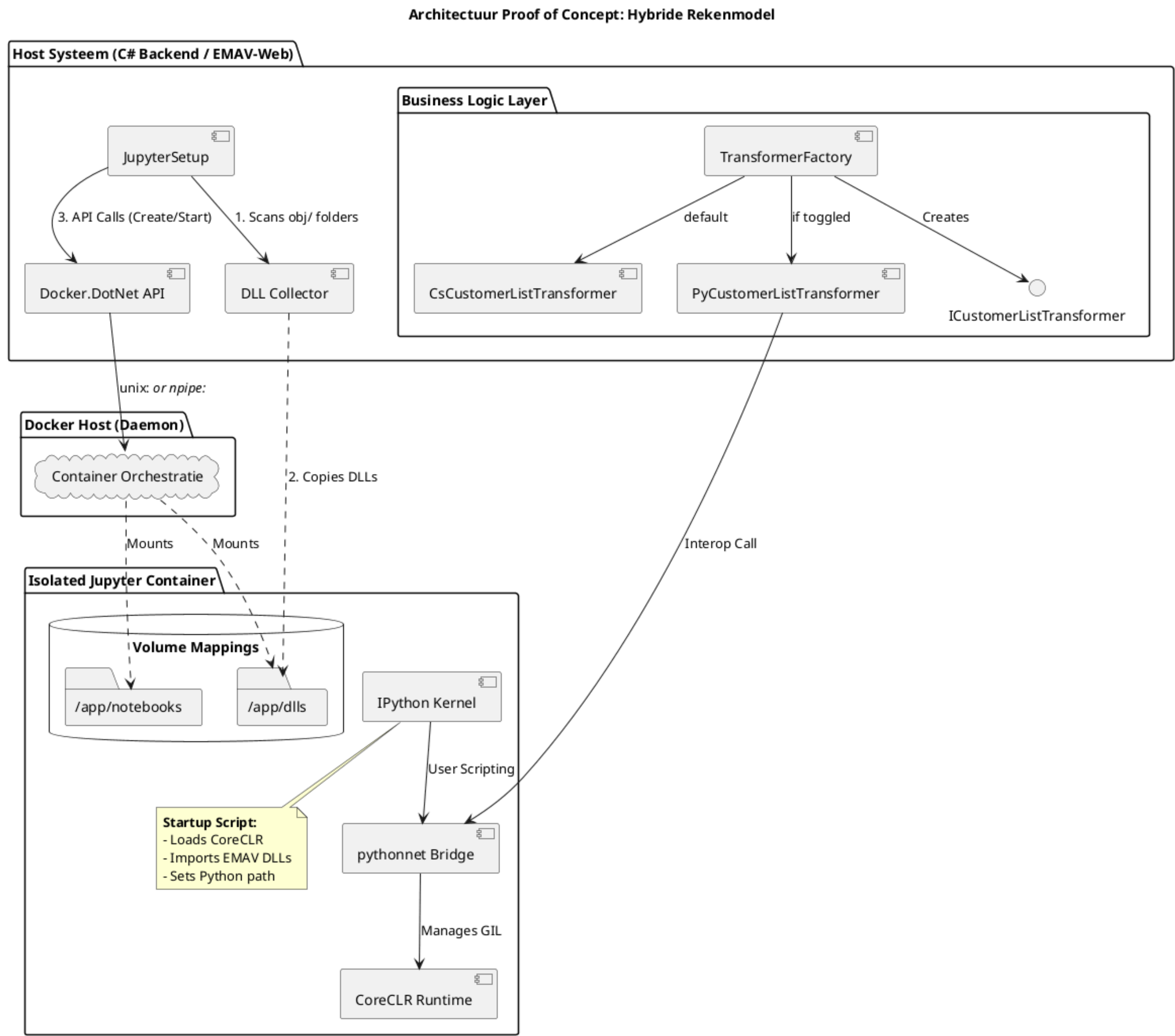
- Bridge-technologie (Python.NET): Gebruik van pythonnet (Python.NET, 2026) om C#-modules (DLL's) direct in Python aan te roepen, zonder de code te herschrijven.
- Interactieve Notebooks (Jupyter): Inzet van Jupyter Notebooks (Project Jupyter, 2025) als interface. Dit combineert uitvoerbare code met rijke tekstuele uitleg (Literate Programming).
- Isolatie & Beveiliging (Docker (Docker Inc., 2026)): De gehele analyse-omgeving is gecontaineriseerd. Dit garandeert reproduceerbaarheid en beschermt de infrastructuur tegen onbedoelde schadelijke code-executie.
- Experimenteer-switch: Ontwikkeling van een mechanisme om veilig te schakelen tussen de officiële C#-berekening en een experimentele Python-versie via een strategy pattern.

3. Proof-of-Concept: Hybride Architectuur

De PoC demonstreert een ecosysteem waarin de robuuste .NET-backend de regie voert over interactieve Jupyter-omgevingen. Dit garandeert een naadloze overgang van ontwikkeling naar wetenschappelijke analyse.

Kernaspecten van de realisatie:

- Geautomatiseerde Orchestratie: De C#-backend beheert via de Docker SDK de levenscyclus van de analyse-omgevingen. Elke onderzoeker krijgt automatisch een identieke, geïsoleerde setup.
- Dynamic DLL Injection: Domeinlogica (C#-libraries) wordt automatisch verzameld en in de container geïnjecteerd. De onderzoeker werkt altijd op de Single Source of Truth van de applicatie.
- Seamless Interop: Dankzij Python.NET (Python.NET, 2026) worden C#-types behandeld als native Python-klassen, wat de technische drempel voor onderzoekers minimaliseert.



Figuur 1: Architecturaal overzicht van de PoC met C#-orchestratie en hybride Python-integratie.

4. Schakelbare Implementatie

Het belangrijkste onderdeel is de mogelijkheid om op laag niveau te wisselen tussen implementaties via een Factory- en Strategy-patroon.

Het mechanisme:

- Dynamische Selectie: Een factory beslist op runtime of een rekenmodule de officiële C#-code of een experimenteel Python-script uitvoert.
- Granulaire Controle: Onderzoekers kunnen per specifieke rekenstap (bijv. “Enterische emissies”) een alternatieve versie testen zonder het hele systeem te beïnvloeden.
- Veiligheid: Door het gebruik van tijdelijke SQLite-kopieën en Docker-isolatie kunnen scenario's getest worden zonder de productie-database te vervuilen.

5. Conclusies & Toekomst

De PoC bewijst dat een hybride strategie de autonomie van onderzoekers verhoogt zonder in te boeten op de softwarekwaliteit van de backend.

Toekomstig onderzoek:

- Structurele inbedding van de switch-logica in alle EMVA-rekenmodules.
- Uitbreiding van de in-notebook validatietools voor automatische vergelijking tussen C# en Python resultaten.
- Verder consulteren met de onderzoekers zelf.
- De PoC volledig uitwerken.
- Een effectieve implementatie van documentatie.